FaceFind

AI-Powered Event Photo Discovery

DEPLOYMENT MANUAL

Document Information	
Document Title	FaceFind Deployment Manual
Version	1.0
Course	CS691 Spring 2026
University	Pace University
Team	Team 7
Authors	Jash Berawala, Niraj Patil, Chetan Patel, Jay Shah
Last Updated	Spring 2026

Section 1 — Audience Definition

1. Audience Definition

We put this manual together so that anyone who needs to get FaceFind running can do it without having to dig through the codebase or ask the team a hundred questions. Here is who we had in mind when writing this:

1.1 Primary Audience

- **System Administrators:** If you are responsible for setting up servers and getting services deployed, this guide walks you through everything you need — from installing dependencies to confirming the app is live.
- **DevOps Engineers:** If you prefer containers, we have a full Docker Compose setup that brings all four services up in a single command. We have made sure the configurations are clean and ready to go.
- **Backend Developers:** If you want to run things locally for development or debugging purposes, the manual setup option (Section 3, Option C) gives you full control over each service.

1.2 Secondary Audience

- **University Evaluators:** Hi! If you are reviewing our project, We would recommend going with Option A or B in Section 3 — they are the least painful paths to getting everything running. The whole flow from clone to running app should take under 10 minutes on a clean machine.
- **QA Testers:** Section 6 has a structured walkthrough of the core user flows with specific things to check at each step. We wrote it based on how we tested the app ourselves during Sprint 3.

NOTE One thing to flag upfront: this is a four-service application (frontend, backend, ML service, and MongoDB). Most issues we ran into during testing came from one service not being fully ready when another tried to connect to it. The health check script in Section 5 helps with that.

Section 2 — Prerequisite Installation

2. Prerequisite Installation

Before you do anything else, make sure these are installed on your machine. We have broken them down by platform so you do not have to go hunting for the right instructions.

2.1 What You Will Need

Dependency	Version	Used For
Node.js	18+	Frontend & Backend
npm	9+ (ships with Node.js)	Package management
Python	3.9+	ML Service
pip	Latest (ships with Python)	Python packages
MongoDB	6.0+	Database
Git	Any	Cloning the repo
Docker Desktop	Latest (optional)	Option B only — skip if not using Docker

2.2 Node.js (v18+)

Windows

1. Go to https://nodejs.org and grab the LTS installer (.msi).
2. Run it and follow the defaults — no need to change anything.
3. Open Command Prompt and confirm it worked:

```
node --version
npm --version
```

macOS

Homebrew is the easiest way to manage this on a Mac:

```
brew install node@18
brew link node@18
```

Or just download the macOS installer directly from https://nodejs.org if you prefer.

Linux (Ubuntu / Debian)

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
node --version && npm --version
```

2.3 Python (v3.9+)

Windows

4. Download the installer from https://python.org/downloads.
5. Important: on the first screen, check 'Add Python to PATH' before clicking Install. Easy to miss, causes headaches if you skip it.
6. Verify it installed correctly:

```
python --version
pip --version
```

macOS

```
brew install python@3.11
```

Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3-pip
python3 --version
```

2.4 MongoDB (v6.0+)

Windows

7. Download MongoDB Community Server from https://mongodb.com/try/download/community.
8. Run the .msi installer, select 'Complete' setup, and let it install MongoDB as a Windows service so it starts automatically.
9. You should not need to do anything else — it runs in the background.

macOS

```
brew tap mongodb/brew
brew install mongodb-community@6.0
brew services start mongodb-community@6.0
```

Linux (Ubuntu)

```
sudo apt-get install -y gnupg curl
curl -fsSL https://pgp.mongodb.com/server-6.0.asc | sudo gpg -o
/usr/share/keyrings/mongodb-server-6.0.gpg --dearmor
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-6.0.gpg
] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/6.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-6.0.list
sudo apt-get update && sudo apt-get install -y mongodb-org
sudo systemctl start mongod && sudo systemctl enable mongod
```

TIP If MongoDB is giving you trouble on any platform, honestly the quickest fix is to just run it in Docker: docker run -d -p 27017:27017 --name facefind-mongo mongo:6. That is what we did during early development and it worked perfectly.

Section 3 — Platform-Specific Deployment Instructions

3. Deployment Instructions

We have three ways to run FaceFind. We would recommend Option A for most people. Option B is the most hands-off. Option C is for anyone who wants full manual control.

3.1 Option A — Automated Setup (Recommended)

NOTE This is what we use day-to-day. It works cleanly on macOS and Linux. Windows users should run it through Git Bash or WSL2.

Step 1 — Clone the Repo

```
git clone https://github.com/JashBerawala/Pace-Project-Team7.git
cd Pace-Project-Team7
```

Step 2 — Run the Setup Script

macOS / Linux

```
chmod +x setup.sh
./setup.sh
```

Windows (Git Bash or WSL2)

```
bash setup.sh
```

Step 3 — Start Each Service

You will need three separate terminal windows for this — one per service. Each one needs to stay open while you are running the app.

Terminal 1 — Backend

```
cd backend
npm run dev
# Running at http://localhost:5001
```

Terminal 2 — ML Service

```
cd ml-service
```

```
uvicorn main:app --reload --port 8000
# Running at http://localhost:8000
# Note: first run downloads InsightFace models (~300 MB) — give it a few minutes
```

Terminal 3 — Frontend

```
cd frontend
npm start
# Running at http://localhost:3000
```

3.2 Option B — Docker Compose (Easiest)

NOTE Docker handles everything — all four services start automatically. You just need Docker Desktop installed and running before you begin.

Step 1 — Clone the Repo

```
git clone https://github.com/JashBerawala/Pace-Project-Team7.git
cd Pace-Project-Team7
```

Step 2 — Build and Start Everything

```
docker-compose up --build
```

Docker builds images for the frontend, backend, ML service, and MongoDB, then starts them all up. Once you see the frontend ready message in the logs, open http://localhost:3000.

Step 3 — Shut It Down

```
docker-compose down
```

3.3 Option C — Manual Setup

Windows-Specific Notes Before You Start

- Use PowerShell or Command Prompt for all commands below.
- When activating the Python virtual environment, use venv\\Scripts\\activate instead of the macOS/Linux version.
- Backslashes in file paths are your friend on Windows.

Step 1 — Start MongoDB

Windows

```
net start MongoDB
```

macOS

```
brew services start mongodb-community@6.0
```

Linux

```
sudo systemctl start mongod
```

Step 2 — Backend

```
cd backend
npm install
cp .env.example .env
# Open .env and double-check the values (see Section 4)
npm run dev
```

Step 3 — ML Service

macOS / Linux

```
cd ml-service
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Windows

```
cd ml-service
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Step 4 — Frontend

```
cd frontend
npm install
npm start
```

Section 4 — Configuration Instructions

4. Configuration Instructions

Most of the configuration happens in a single .env file in the backend folder. Here is what you need to know.

4.1 Backend Environment Variables

Start by copying the example file:

```
cp backend/.env.example backend/.env
```

Variable	Default	What It Does
PORT	5001	The port the backend API listens on
MONGODB_URI	mongodb://localhost:27017/facefind	MongoDB connection string — change this for remote or Atlas deployments
ML_SERVICE_URL	http://localhost:8000	Where the backend looks for the ML service
NODE_ENV	development	Set to production when deploying live

HEADS UP Please do not commit your .env file. It is already in .gitignore but worth double-checking before you push, especially if you add any API keys or credentials later.

4.2 MongoDB Connection

The default config assumes MongoDB is running locally. If you are pointing to MongoDB Atlas or a remote instance, update MONGODB_URI in your .env:

```
# Local MongoDB (default, works out of the box)
MONGODB_URI=mongodb://localhost:27017/facefind

# MongoDB Atlas (cloud)
MONGODB_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/facefind
```

4.3 ML Service — Face Matching Threshold

The ML service does not have its own config file. The one thing worth knowing is the similarity threshold in ml-service/main.py. We tuned it to 0.4 during Sprint 3, which gave us the best balance of accuracy without too many false positives:

```
# ml-service/main.py
SIMILARITY_THRESHOLD = 0.4  # Lower = stricter matching, Higher = more permissive
```

4.4 Port & Firewall Settings

For local development, you do not need to change anything. If you are deploying to a server, here is what needs to be accessible:

Port	Service	Who Should Access It
3000	React Frontend	Public (or reverse-proxied via 80/443)
5001	Node.js Backend	Internal only (called by the frontend)
8000	Python ML Service	Internal only — do not expose this publicly
27017	MongoDB	Internal only — never expose this publicly

Section 5 — Deployment Scripts & Code Snippets

5. Deployment Scripts & Code Snippets

5.1 What setup.sh Does

The setup script automates all the dependency installation so you do not have to run npm install and pip install manually across three folders. Here is what it does under the hood:

```
#!/bin/bash
# FaceFind Automated Setup Script

echo "Installing backend dependencies..."
cd backend && npm install && cp .env.example .env && cd ..

echo "Installing frontend dependencies..."
cd frontend && npm install && cd ..

echo "Setting up Python virtual environment for ML service..."
cd ml-service && python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt && deactivate && cd ..

echo "All done! See Section 3 for the start commands."
```

5.2 docker-compose.yml Overview

For those using Option B, here is what the compose file sets up — all four services wired together with the right environment variables and dependencies:

```
version: '3.8'
services:
  mongo:
    image: mongo:6
    ports: ['27017:27017']

  backend:
    build: ./backend
  ports: ['5001:5001']
    depends_on: [mongo]
    environment:
   - MONGODB_URI=mongodb://mongo:27017/facefind
      - ML_SERVICE_URL=http://ml-service:8000

  ml-service:
    build: ./ml-service
    ports: ['8000:8000']

  frontend:
```

```
    build: ./frontend
    ports: ['3000:3000']
    depends_on: [backend]
```

5.3 Health Check Script

We wrote this quick script to save time when verifying everything is up. Run it after starting the services — it pings each one and tells you what is responding:

```
#!/bin/bash
# health_check.sh

check() {
  curl -sf $2 > /dev/null && echo "[OK]  $1" || echo "[FAIL] $1 not responding at
$2"
}

check "Frontend"    "http://localhost:3000"
check "Backend API" "http://localhost:5001/api/events"
check "ML Service"  "http://localhost:8000/docs"
```

```
chmod +x health_check.sh
./health_check.sh
```

Section 6 — Testing & Troubleshooting

6. Testing & Troubleshooting

6.1 End-to-End Test Walkthrough

Once everything is running, go through these tests in order. This is basically the same checklist we used ourselves at the end of Sprint 3.

Test 1 — Confirm Services Are Up

10. Open http://localhost:3000 — the FaceFind landing page should load cleanly.
11. Go to http://localhost:3000/admin — the Admin Dashboard should appear.
12. Go to http://localhost:8000/docs — you should see the FastAPI Swagger UI for the ML service.

Test 2 — Create an Event

13. In the Admin Dashboard, click '+ New Event' and fill in a name and date.
14. Hit Create — the new event should show up in the list with a unique code and a QR code button.

Test 3 — Upload Photos and Watch Processing

15. Click 'Manage →' on the event you just created.
16. Drag and drop at least one photo into the upload area.
17. Watch the real-time progress indicators. Green means the ML service has successfully processed the photo and stored the face embeddings. If it stays yellow or turns red, check the ML service terminal for errors.

Test 4 — Guest Face Matching

18. Open the guest link in a new tab (or scan the QR code on your phone).
19. Upload a selfie of someone who appears in one of the photos you uploaded.
20. Click 'Find My Photos' — the matching results should appear within a few seconds.

TIP When testing face matching, use a well-lit selfie with a clear frontal face. The InsightFace model we are using needs a minimum face size of about 40x40 pixels to work reliably. Blurry or heavily angled photos will not match well.

6.2 Quick API Tests with curl

If you want to verify the backend directly without going through the UI:

```
# Check if the backend is responding
```

```
curl http://localhost:5001/api/events

# Create a test event
curl -X POST http://localhost:5001/api/events \
  -H 'Content-Type: application/json' \
  -d '{"name": "Test Event", "date": "2026-05-10"}'
```

6.3 Common Issues and How to Fix Them

Issue: ML service crashes on startup

This usually means the InsightFace models did not download automatically. Run this to trigger it manually:

```
python3 -c "from insightface.app import FaceAnalysis; app = FaceAnalysis();
app.prepare(ctx_id=0)"
```

HEADS UP The model download is about 300 MB and needs a stable internet connection. If you are behind a corporate firewall, it may block the download. Try from a different network if you keep hitting failures.

Issue: Backend cannot connect to MongoDB

First, check if MongoDB is actually running:

```
mongosh --eval "db.adminCommand('ping')"
```

If that fails, start it for your platform:

```
# Windows
net start MongoDB

# macOS
brew services start mongodb-community@6.0

# Linux
sudo systemctl start mongod
```

Issue: 'Address already in use' / port conflict

Another process is sitting on the port. Here is how to clear it:

macOS / Linux

```
lsof -ti:3000 | xargs kill    # Frontend
lsof -ti:5001 | xargs kill    # Backend
lsof -ti:8000 | xargs kill    # ML Service
```

Windows

```
netstat -ano | findstr :5001
taskkill /PID <the_pid_number> /F
```

Issue: npm install fails with permission errors

This pops up sometimes on macOS or Linux. Fix it with:

```
sudo chown -R $(whoami) ~/.npm
```

On Windows, right-click your terminal and choose 'Run as Administrator'.

Issue: Docker containers exit immediately

21. Make sure Docker Desktop is actually open and running before you run docker-compose.
22. Check you have enough free disk space — the build needs a few GB.
23. If something looks corrupted, do a clean rebuild:

```
docker-compose down --volumes
docker-compose up --build --force-recreate
```

Issue: Face matching returns no results

- Make sure the event photos finished processing — all progress dots in the Admin panel should be green before you test matching.
- Try a different, clearer selfie. Lighting and angle make a big difference with face recognition.
- Check the ML service logs for the similarity scores it is computing. If they are all below 0.4, try lowering the threshold in ml-service/main.py slightly.
- Confirm the ML service is still running at http://localhost:8000.

6.4 Service URLs at a Glance

Service	URL	Notes
Main App	http://localhost:3000	Start here
Admin Dashboard	http://localhost:3000/admin	Photographer panel
Backend API	http://localhost:5001/api	REST API base
ML Docs (Swagger)	http://localhost:8000/docs	Useful for debugging ML calls directly